

ROBOTIC ARM FOR PICK AND PLACE

A MINI-PROJECT REPORT

submitted by

**ROHITH R NAIR (REG NO: MGP22URB080),
NEVIN SANUSH (REG NO: MGP22URB075),
JOHAN MATHEWS ABRAHAM (REG NO: MGP22URB058),
SOORAJ K S (REG NO: MGP22URB093)**

*in partial fulfilment of the requirements for the
award of the degree of*

Third Year B.Tech
(ROBOTICS AND AUTOMATION ENGINEERING)
2022-26



**DEPARTMENT OF ELECTRONICS ENGINEERING
SAINTGITS COLLEGE OF ENGINEERING
(AUTONOMOUS)**

KOTTAYAM

MARCH 2025

DECLARATION

We hereby declare that the project report entitled “ROBOTIC ARM FOR PICK AND PLACE”, submitted for the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology of the A P J Abdul Kalam Technological University, Kerala is a bonafide work done by us under the supervision of Dr. Riboy Cheriyan. This submission represents our ideas in our own words and where ideas or words of others have been included. We have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to academic honesty and integrity ethics and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma, or similar title of any other University.

Place: Pathamuttom

Date:

Rohith R Nair

Nevin Sanush

Johan Mathews Abraham

Sooraj K S

ROBOTICS AND AUTOMATION ENGINEERING
SAINTGITS COLLEGE OF ENGINEERING (AUTONOMOUS)
PATHAMUTTOM, KOTTAYAM, KERALA - 686532



CERTIFICATE

This is to certify that the mini project report entitled “ **ROBOTIC ARM FOR PICK AND PLACE** ” submitted by **Rohith R Nair (MGP22URB080)**, **Nevin Sanush (MGP22URB075)**, **Johan Mathews Abraham (MGP22URB058)**, **Sooraj K S (MGP22URB093)** , to the A P J Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Robotics and Automation Engineering is a bonafide record of the project work carried out. This report in any form has not been submitted to any other University or Institute for any purpose.

Dr. Riboy Cheriyan
Project co-ordinator
Dept. Electronics Engineering

Er. Bini Thomas
Project Guide
Dept. of Electronics Engineering

Place: Kottayam
Date:

Dr. Sreekala K S
Head of the Department
Dept. of Electronics Engineering

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without acknowledging the people whose constant guidance and encouragement have crowned all the efforts with success. It would not be possible to complete the project without their valuable help, cooperation, and guidance.

First, I thank God Almighty, for lending the talent to do this project and completing the work successfully.

I take this opportunity to thank **Dr . Sudha T**, Principal, Saintgits College of Engineering, Pathamuttom for providing the best facilities and environment to prepare and present this project. I am grateful for his support and cooperation. I also thank **Dr . Sreekala K S**, Head of the Department, of Electronics Engineering for her advice and encouragement.

I express my sincere gratitude to **Er . Bini Thomas** (Mini Project Coordinator), and **Dr. Riboy Cheriyan** (Project Guide), for their helpful guidance, suggestions, and encouragement in the completion of the project. Finally, I special thanks to our other staff members and all my beloved friends for their timely help.

ABSTRACT

KEYWORDS: Robotic Arm, Pick And Place, Inverse Kinematics, Servo Motors, Stepper Motors, Arduino Mega

The objective of this project is to design and develop a robotic arm capable of performing pick-and-place tasks with precision and efficiency using inverse kinematics. This robotic arm is intended to automate repetitive tasks in industries such as manufacturing, warehousing, and assembly lines, where human labor can be replaced with robots to improve productivity and safety. The robotic arm will be equipped with various sensors, actuators, and controllers to ensure accurate object detection, positioning, and manipulation. The control system will be programmable, enabling the arm to adapt to different tasks and environments. To enhance the functionality, the robotic arm will be designed to pick objects of various shapes and sizes, while ensuring safety features to avoid collisions with humans or other machines in its workspace. The arm will be powered by servomotors and stepper motors controlled by microcontrollers such as Arduino Mega. The system will be monitored and controlled via a user-friendly interface, allowing operators to customize the task parameters, monitor the arm's status, and receive alerts in case of malfunctions. The data generated during the operation will be logged for future analysis to further optimize the system's performance. This project aims to reduce human effort in industrial settings, improve task precision, and contribute to the automation of modern industries. The system will be tested for its accuracy, reliability, adaptability and to minimize downtime.

Contents

CERTIFICATE	.	ii
ACKNOWLEDGEMENT		iv
ABSTRACT	.	v
LIST OF FIGURES	.	ix
1 INTRODUCTION		2
1.1 Design	.	3
1.2 Decision Making	.	4
1.2.1 Requirement	.	4
1.2.2 Research and Analysis		5
1.2.3 Conceptual Design	.	5
1.2.4 Motor and Controller Selection	.	5
1.2.5 Mechanical Design	.	5
2 LITERATURE SURVEY		7
2.1 Noor Saeed , " <i>Forward and Inverse Kinematic Analysis of Robots</i> " published by LAP LAMBERT Academic Publishing in 2020		7
2.2 Dilip Kumar Pratihari, " <i>Manipulators, Wheeled and Legged Robots</i> " published by Narosa Publishing House in 2024		7
2.3 Jiawei Zhang, " <i>Robot Kinematics: Motion, Kinematics and Dynamics</i> " (2022)		8
2.4 Sitangshu Sekhar Biswas, " <i>Hand Book of Bi-Polar Stepper Motor Control</i> " (2021)		8

3	CHALLENGES WE FACED WHILE MAKING ROBOTIC ARM FOR PICK AND PLACE	9
3.1	Mechanical problems:	9
3.2	Design-related problems:	9
3.3	Programming-related problems:	9
4	SYSTEM MODEL	10
4.1	Arduino Mega 2560	10
4.1.1	Architecture Of Arduino Mega 2560	11
4.2	RAMPS 1.4 Shield	12
4.3	Stepper Motors (Nema 14,17,23)	13
4.4	Servo Motor (MG995)	14
4.5	Motor Driver (TB6560)	15
5	IMPLEMENTATION	17
5.1	Hardware	17
5.2	Arduino IDE	17
5.3	Sensor Integration	17
5.4	Testing	17
6	SOFTWARE AND HARDWARE	18
6.1	Software Requirements	18
6.2	System Requirement	18
6.3	Hardware Requirements	18
7	EXPERIMENTAL ANALYSIS AND RESULT	20
7.1	Result	20
7.2	Performance Evaluation	20
7.3	Arduino Code	20
7.4	Python Code For Inverse Kinematics	25
8	CONCLUSION AND FUTURE SCOPE	30
9	REFERENCES	31

List of Figures

1.1	Design	.	3
1.2	Reference Framework	.	4
4.1	System Model	.	10
4.2	Arduino Mega 2560		12
4.3	Ramp Shield 1.4		13
4.4	General Stepper Motor	.	14
4.5	MG995 Servo Motor	.	15
4.6	TB6560 Motor Driver	.	16
6.1	PLA+ Parts		19

CHAPTER 1

INTRODUCTION

Efficiently moving objects from one location to another is a fundamental requirement in many industrial and commercial processes. However, performing repetitive pick-and-place tasks manually can be time-consuming, labor-intensive, and prone to human error. To address these challenges, the development of an automated pick-and-place robotic manipulator offers a streamlined, precise, and reliable solution.

This report focuses on the design, construction, and functionality of a 3D-printed 5-DOF robotic arm dedicated to pick-and-place operations. The system is controlled by an Arduino Mega paired with a RAMPS 1.4 shield, and its movement is driven by a combination of stepper and servo motors. These components work together to enable precise motion control and accurate object handling. The primary objective of the robotic arm is to execute accurate and repeatable pick-and-place tasks in a safe and efficient manner, thereby reducing manual labor and increasing throughput.

The robotic manipulator is designed to operate autonomously or under direct human supervision. It incorporates advanced control algorithms, mechanical actuation systems, and sensing technologies to grasp objects, navigate through defined paths, and accurately place items at designated locations. Key topics covered in this report include the arm's kinematic structure, drive mechanisms, control system integration, and the software/hardware interfaces that coordinate its movements.

By deploying this 5-DOF pick-and-place robotic arm—controlled by an Arduino Mega with a RAMPS 1.4 shield and powered by stepper and servo motors—organizations can enhance operational efficiency, improve accuracy in repetitive tasks, and reduce human involvement in hazardous or monotonous operations. Its ability to automate material handling leads to cost savings, improved process reliability, and enhanced product quality. In summary, the pick-and-place robotic manipulator presented in this report represents a significant advancement in automated material handling. Its modular design, precision control, and adaptability make it a valuable tool for modern automation applications. The subsequent sections will provide a comprehensive analysis of the robot's design, operation, and potential applications, highlighting its impact on industry and its promise for future development.

1.1 Design

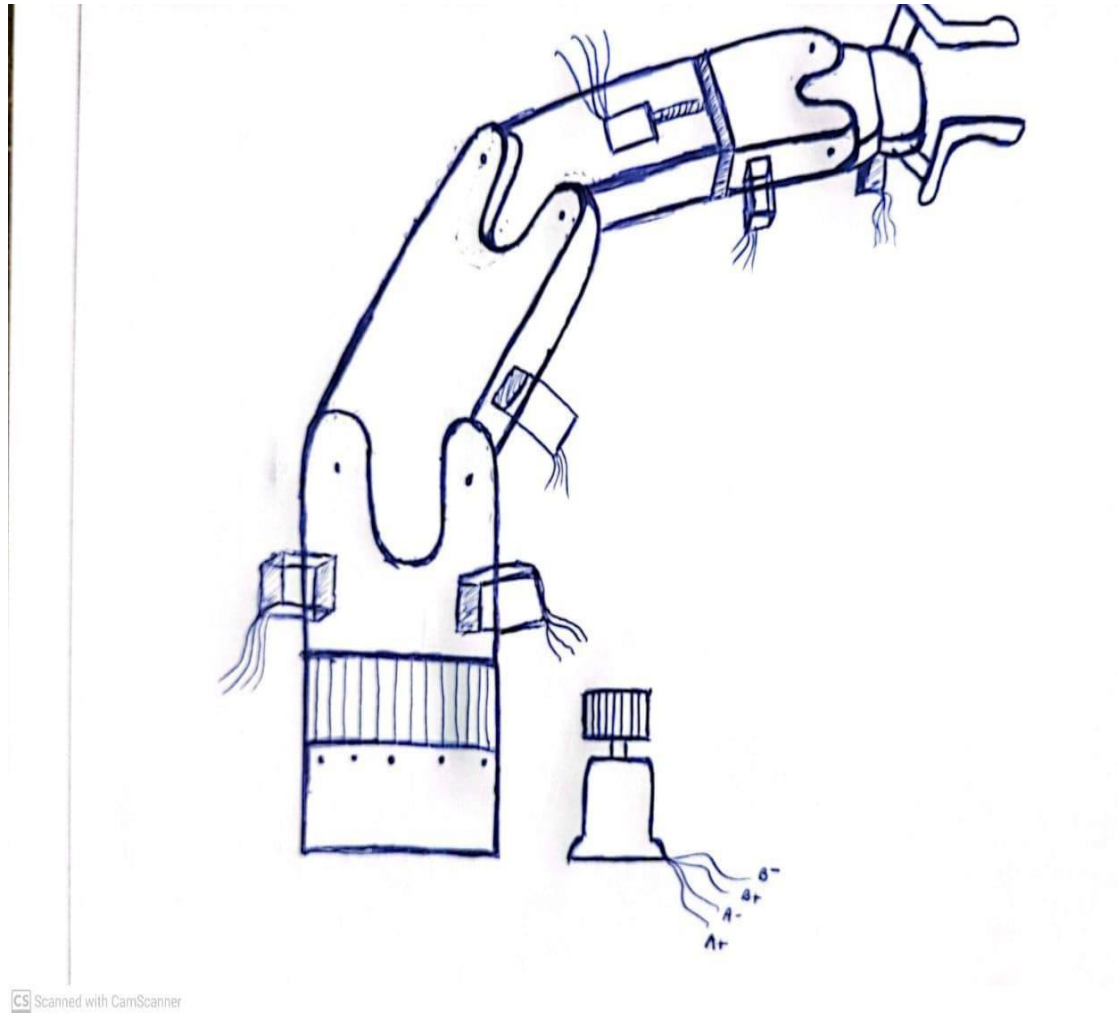


Figure 1.1

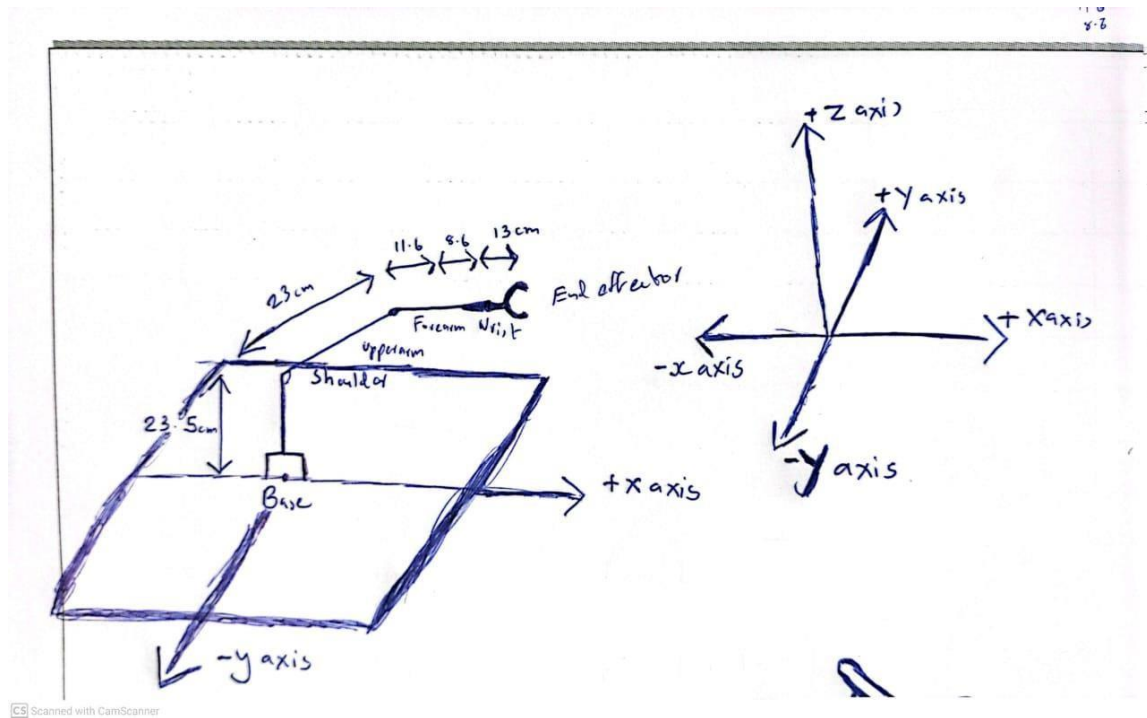


Figure 1.2

1.2 Decision Making

The decision-making process of the pick-and-place robotic arm involves several key components and control algorithms working together to enable precise and efficient operation. Here are some important aspects of the decision-making process:

1.2.1 Requirement

A 5-DOF robotic arm has various applications in industries where automation and precise object manipulation are required. It is widely used in industrial automation for manufacturing and assembly lines, reducing human effort and increasing efficiency. In warehousing and logistics, the robotic arm plays a crucial role in sorting, packaging, and organizing products, improving workflow optimization. It also serves as an educational and research tool, helping students and researchers understand kinematics, motion control, and robotics.

1.2.2 Research and Analysis

Research on existing robotic arm designs and inverse kinematics techniques was conducted, focusing on different types of pick-and-place applications, motion planning methods, and mechanical structures. The feasibility of using the Arduino Mega 2560 with a RAMPS 1.4 shield for motor control was analyzed to ensure precise movement execution. Additionally, stepper motors and servo motors were studied to determine their suitability for different joints of the robotic arm.

1.2.3 Conceptual Design

Based on research, multiple conceptual designs for the robotic arm were generated. Key factors considered included the range of motion for each joint, end-effector positioning accuracy, stepper and servo motor selection for controlled movement, and the integration of inverse kinematics for smooth trajectory planning. The design aimed to achieve a balance between mechanical rigidity and flexibility to perform pick-and-place tasks effectively.

1.2.4 Motor and Controller Selection

The robotic arm is driven by stepper motors and a servo motor for precise movement. The selected components include the Arduino Mega 2560 as the main controller, the RAMPS 1.4 Shield for motor control, TB6560 stepper motor drivers for precise actuation, Stepper Motors (NEMA 17 & NEMA 23) for joint movement, and a Servo Motor for end-effector control. These components ensure accurate motion execution and smooth operation of the robotic arm.

1.2.5 Mechanical Design

The mechanical design was developed considering link lengths, weight distribution, and structural rigidity. Factors such as joint placement for maximum reach, durability of materials for long-term performance, and a compact design for efficient workspace utilization were carefully analyzed. The structural components were designed to withstand the load while maintaining the necessary flexibility to operate in different environments.

- **Electronics and Control System**

The electronics and control system were designed to enable smooth operation and precise control of the robotic arm. The Arduino Mega 2560 serves as the central controller, while the RAMPS 1.4 Shield facilitates motor control. TB6560 stepper motor drivers ensure precise movement of stepper motors, while a 12V power system supports both the stepper motors and the servo motor. The inverse kinematics calculations are implemented using Python, allowing for precise computation of joint angles to achieve the desired end-effector position.

- **Integration and Testing**

All components were integrated, and extensive testing was conducted to evaluate the robotic arm's ability to execute pick-and-place tasks accurately. The testing process involved verifying the movement of each joint, ensuring the robotic arm followed predefined trajectories using inverse kinematics, and assessing its stability under different payloads. Various test scenarios were implemented to measure precision, repeatability, and reliability in object manipulation.

- **Refinement and Optimization**

The performance of the robotic arm was assessed, and multiple refinements were made to enhance its capabilities. Motor response was improved for smoother motion, inverse kinematics algorithms were optimized for real-time control, and power consumption was reduced while maintaining accuracy. Future developments will focus on integrating a camera to enable autonomous object recognition and decision-making, further enhancing the potential applications of the robotic arm in industrial and research environment.

CHAPTER 2

LITERATURE SURVEY

2.1 Noor Saeed , "*Forward and Inverse Kinematic Analysis of Robots*" published by LAP LAMBERT Academic Publishing in 2020

Under the title, "Forward and Inverse Kinematics," this book offers a mathematical approach to understanding robot kinematics, providing equations for converting between joint space and Cartesian positions. It includes software tools developed in Visual Basic and MATLAB for modeling and simulating various robot configurations.

2.2 Dilip Kumar Pratihari, "*Manipulators, Wheeled and Legged Robots*" published by Narosa Publishing House in 2024

Under the title, "Effective Power Transmission - Fundamentals of Robotics and AI", this comprehensive text delves into the principles of robotics, including discussions on power transmission mechanisms essential for robotic manipulators. It covers topics like robot dynamics and motion planning, providing insights into effective power transmission in robotic systems.

2.3 Jiawei Zhang, "*Robot Kinematics: Motion, Kinematics and Dynamics*" (2022)

Under the title, "Managing Center of Mass", this tutorial covers advanced topics in robot kinematics, including motion and dynamics, which are essential for understanding and managing the center of mass in robotic systems.

2.4 Sitangshu Sekhar Biswas, "*Hand Book of Bi-Polar Stepper Motor Control*" (2021)

Under the title, "Bi-Polar Stepper Motor Control" This comprehensive text delves into the principles of robotics, including discussions on power transmission mechanisms essential for robotic manipulators. It covers topics like robot dynamics and motion planning, providing insights into effective power transmission in robotic systems.

CHAPTER 3

CHALLENGES WE FACED WHILE MAKING ROBOTIC ARM FOR PICK AND PLACE

3.1 Mechanical Problems:

- The initial structure was not sturdy enough to support the robotic arm's weight, requiring modifications in material selection.
- Some stepper motors lacked sufficient torque for precise movement, leading to issues in handling payloads effectively.
- The planetary stepper motor required additional fine-tuning to achieve smooth motion.
- Proper alignment of linkages was difficult, causing instability during pick-and-place operations.

3.2 Design-Related Problems:

- Determining the optimal structure for the robotic arm was challenging, as different configurations affected stability and flexibility.
- The mounting of the RAMPS 1.4 shield and TB6560 drivers required adjustments to ensure efficient wiring and heat dissipation.

3.3 Programming-Related Problems:

- Implementing inverse kinematics calculations accurately for real-time control of the robotic arm required significant debugging.
- Synchronizing multiple stepper motors while avoiding unwanted vibrations or delays was complex.
- Wireless communication using the built-in ESP8266 on the Arduino Mega 2560 required additional coding for remote control and monitoring.
- Fine-tuning the motion planning for smooth and precise movements was a time-consuming process.

CHAPTER 4

SYSTEM MODEL

A system model for a robotic arm for pick and place detection robot typically involves several components working together to perform motor actuation and movement. Here is a simplified representation of the system model:

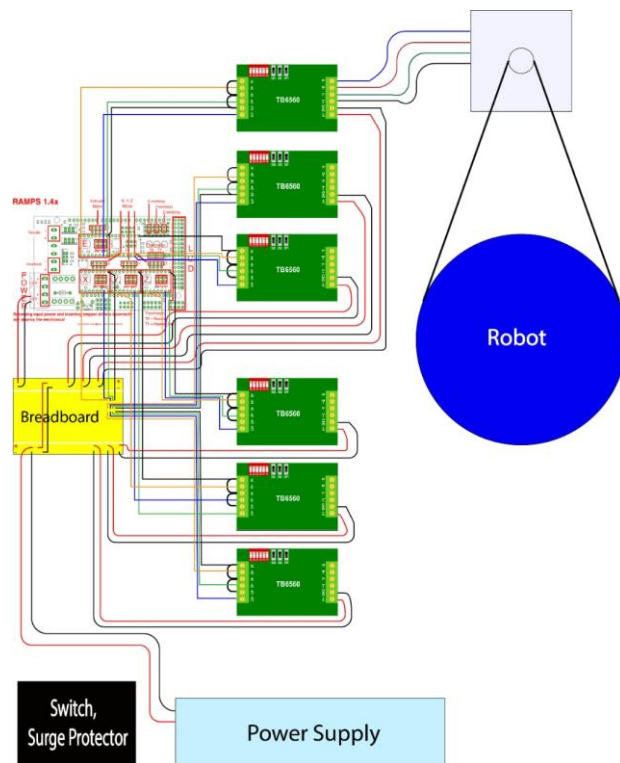


Figure 4.1: System Model

4.1 Arduino Mega 2560

The Arduino Mega 2560 is a popular microcontroller board based on the ATmega2560 microcontroller. It is widely used in various electronic projects and prototyping due to its higher number of input/output pins, increased memory, and suitability for complex applications like robotics and automation.

4.1.1 Architecture Of Arduino Mega 2560

Microcontroller:

The Arduino Mega 2560 is powered by the ATmega2560 microcontroller, which operates at 16 MHz. It has 256KB of flash memory for program storage, 8KB of SRAM, and 4KB of EEPROM for data storage.

Digital I/O Pins:

The board has 54 digital input/output (I/O) pins, out of which 15 can be used as Pulse Width Modulation (PWM) outputs. These pins can interface with various external devices such as sensors, actuators, LEDs, and displays.

Analog Inputs:

The Arduino Mega 2560 features 16 analog input pins, labeled A0 to A15. These pins can be used to read analog sensor values or voltage levels ranging from 0 to 5 volts.

Power Supply:

The board can be powered through a USB connection or an external power supply. The operating voltage is 5V, and it can also be powered through the Vin pin, accepting an external power source within the recommended voltage range of 7V to 12V.

Communication:

The Arduino Mega 2560 includes four hardware serial ports (UARTs) for communication with multiple devices. It features a USB connection that enables programming and communication with a computer, using the ATmega16U2 microcontroller as a USB-to-serial converter. Additionally, it has an ICSP (In-Circuit Serial Programming) header for advanced programming and communication options.

Programming:

The Arduino Mega 2560 is programmed using the Arduino programming language, which is based on C/C++. The Arduino IDE (Integrated Development Environment) provides a user-friendly interface for writing, compiling, and uploading code to the board.

Expansion:

The Arduino Mega 2560 is compatible with a wide range of shields and modules, which are add-on boards that provide additional functionality such as Ethernet connectivity, wireless communication (Bluetooth, Wi-Fi), motor control, and more. These shields can be easily connected to the Arduino Mega 2560, expanding its capabilities.

Open-Source:

The Arduino Mega 2560 is an open-source hardware platform, meaning that its design files, schematics, and firmware are freely available. This allows users to modify and customize the board according to their specific needs.

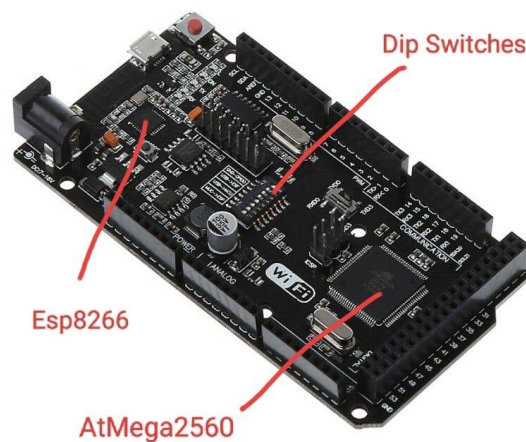


Figure 4.2: Arduino Mega 2560

4.2 RAMPS 1.4 Shield

The RAMPS 1.4 (RepRap Arduino Mega Pololu Shield) is a widely used expansion board designed to work with the Arduino Mega 2560 for controlling stepper motors and other components in CNC machines, 3D printers, and robotic systems. It serves as an interface between the Arduino Mega 2560 (with built-in ESP8266) and external hardware like stepper drivers (TB6560), power supplies, and sensors.

RAMPS 1.4 supports up to five stepper motors, making it ideal for multi-axis motion control in robotic arms. It includes MOSFET outputs for high-power components like heaters, fans, and actuators, allowing precise control of multiple devices. The shield operates on 12-24V DC, and in this project, a 24V SMPS is stepped down to 12V using an XL4016 DC-DC converter to ensure stable power distribution.

Its modular and expandable design makes it suitable for automation tasks like pick-and-place operations, spray painting, sorting, welding, and industrial manufacturing. Additionally, the ESP8266 on the Arduino Mega 2560 enables wireless communication, allowing remote monitoring and control. This makes RAMPS 1.4 a versatile and reliable component for advanced robotics applications.

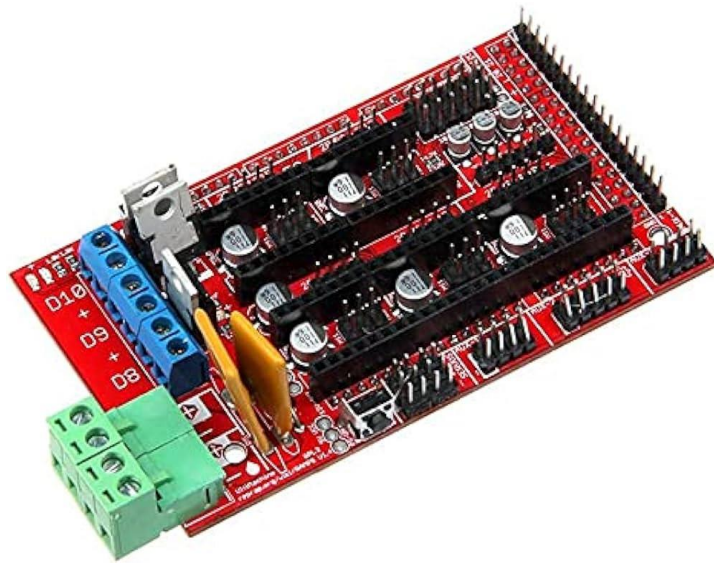


Figure 4.3: Ramp shield 1.4

4.3 Stepper Motors (Nema 14,17,23)

Stepper motors are precise and controlled electromechanical devices used in robotics, CNC machines, and automation applications. They move in discrete steps, providing accurate position control without requiring feedback sensors. This project utilizes six stepper motors of different specifications to drive the robotic arm effectively.

1. High Torque & Precision:

- Stepper motors provide high torque at low speeds, making them ideal for robotic arm movements, pick-and-place operations, and industrial applications.
- They move in fixed step angles, ensuring precise positioning without the need for encoders.

2. Controlled Speed & Positioning:

- Unlike DC motors, stepper motors operate with precise step-based movement, making them suitable for applications that require accurate rotation and repeatability.
- The robotic arm utilizes microstepping for smoother and more controlled motion.

3. Efficiency in Motion Control:

- Stepper motors work efficiently with pulse-based control, allowing speed and torque adjustments through stepper motor drivers (TB6560).
- They do not require complex feedback systems, making them cost-effective and reliable.

4. Compact & Versatile Design:

- The stepper motors used in this project vary in size and torque capacity, ensuring optimal performance for different robotic arm joints.
- The combination of NEMA 17, NEMA 23, NEMA 14, and a planetary stepper motor provides flexibility for various robotic tasks.

5. Types of Stepper Motors Used in the Project:

- 2 × NEMA 17 – Used for lightweight movements and positioning.
- 2 × NEMA 23 – Provides high torque for heavy lifting and major arm movement.
- 1 × NEMA 14 – Handles small, precise adjustments.
- 1 × NEMA 17 Planetary Stepper Motor – Offers high torque with smooth motion for fine movements.

6. Durability & Reliability:

- Stepper motors are highly reliable as they have no brushes or commutators, reducing maintenance.
- Their design ensures long operational life even under continuous use in industrial environments.

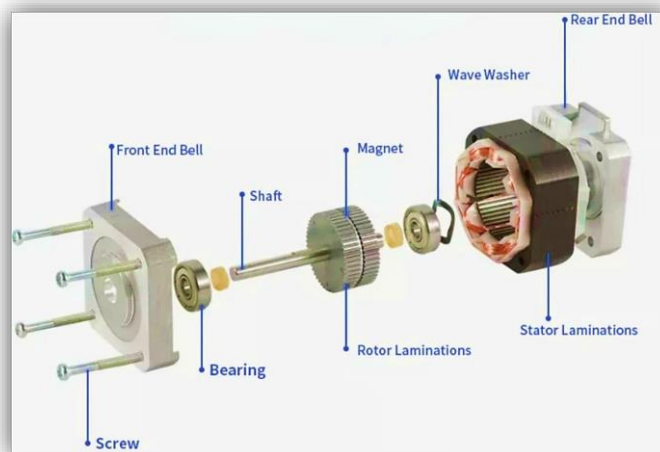


Fig 4.4 General Stepper Motor

4.4 Servo Motor (MG995)

The MG995 is a high-torque metal gear servo motor widely used in robotics and automation due to its durability and precision. It operates using PWM signals for accurate position control, making it ideal for robotic arms and motion-based applications.

1. High Torque Output:

- Provides a torque of 9.4 kg/cm at 6V, suitable for lifting and gripping tasks.

2. **Rotational Range:**
 - Supports 0° to 180° movement with precise angle control.
3. **Metal Gear Construction:**
 - Durable metal gears ensure long life and reliability under heavy loads.
4. **Operating Voltage and Speed:**
 - Operates between 4.8V to 7.2V, with an optimal speed of ~0.2 sec per 60° at 6V.
5. **Application in the Project:**
 - Used in the pick-and-place robotic arm for gripping and object manipulation.
 - Controlled via Arduino Mega 2560 for precise movement coordination.

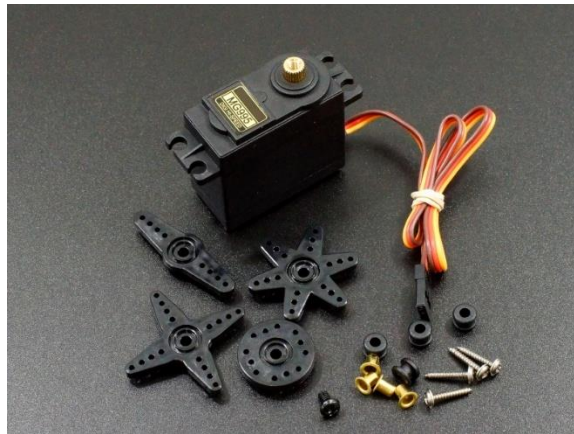


Figure 4.5: MG995 Servo Motor

4.5 Motor Driver (TB6560)

The TB6560 is a stepper motor driver commonly used for controlling high-power stepper motors in CNC machines, robotic arms, and automation projects. It provides precise control over stepper motors with microstepping capability and current regulation.

1. **Motor Configuration:**
 - The TB6560 is designed for bipolar stepper motors and supports full-step, half-step, quarter-step, and eighth-step modes for smooth motion.
2. **Maximum Current:**
 - The driver can handle up to 3.5A per phase with adjustable current settings to match the motor's requirements.
3. **Voltage Range:**
 - The TB6560 operates within a 10V to 35V power range, making it suitable for 12V stepper motors used in the robotic arm.

4. Control Inputs:

- Uses Pulse (PUL), Direction (DIR), and Enable (ENA) signals to control the movement of stepper motors from the Arduino Mega 2560.

5. Built-in Protection:

- Equipped with thermal shutdown, overcurrent protection, and undervoltage lockout to prevent damage during operation.

6. Microstepping Capability:

- Supports 1/1, 1/2, 1/4, and 1/8 step modes, allowing smoother motion and better positioning accuracy for the 6 stepper motors in the robotic arm.

7. Heat Dissipation:

- Includes a built-in heatsink to prevent overheating during extended operations. External cooling fans may be used for additional heat dissipation.

8. Compatibility:

- Compatible with TTL 5V logic signals, allowing direct interfacing with the Arduino Mega 2560 for motion control.

9. Application in the Project:

- The TB6560 drivers are used to control 6 stepper motors (2 NEMA 17, 2 NEMA 23, 1 NEMA 14, and 1 NEMA 17 planetary stepper motor) in the robotic arm.
- The RAMPS 1.4 shield acts as an interface between the Arduino Mega 2560 and TB6560 drivers, ensuring smooth operation.
- The motors are powered through a 24V SMPS, stepped down to 12V using an XL4016 DC-DC converter to match the motor driver requirements.

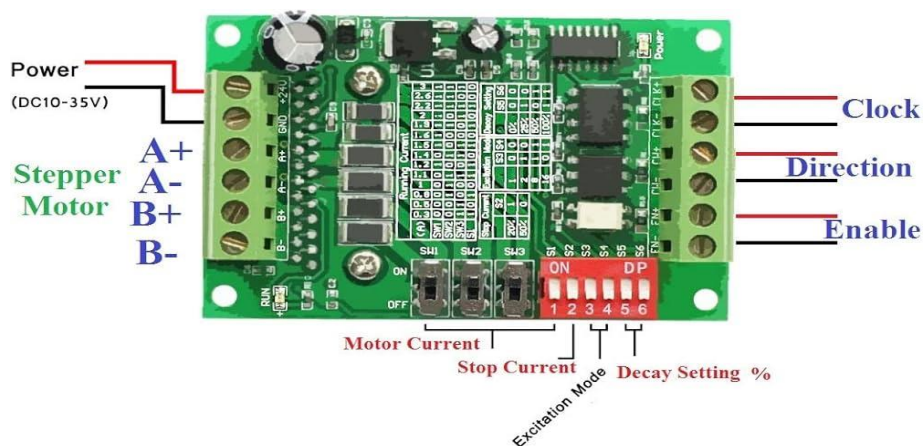


Figure 4.6: TB6560 Motor driver

CHAPTER 5

IMPLEMENTATION

Implementing a 5-DOF pick-and-place robotic arm involves a combination of hardware and software development:

5.1 Hardware

Assemble the physical structure of the robotic arm using 3D-printed parts for the links, along with the base board. Install the necessary actuators including stepper motors for joint movement and a servo motor for the end-effector. Integrate the Arduino Mega 2560, RAMPS 1.4 shield, and TB6560 stepper drivers to ensure precise control of all motors.

5.2 Arduino IDE Installation

Install the Arduino IDE (version 2.1.0 or later) on your computer and download the necessary libraries (such as AccelStepper and Servo) to support motor control and communication with the Arduino Mega 2560.

5.3 Software Integration and Inverse Kinematics

Develop and integrate the Arduino code to control the motor movements, including emergency stop handling and gripper control. Additionally, implement Python-based inverse kinematics algorithms to calculate the required joint angles for precise pick-and-place operations. This integration allows the robotic arm to move accurately along predetermined trajectories.

5.4 Testing

Test the implemented functionality of the pick-and-place robotic arm. This includes verifying the movement accuracy of the stepper motors and the servo-controlled gripper, ensuring that the inverse kinematics calculations produce correct joint angle commands, and confirming that the overall system operates reliably under simulated and real conditions.

CHAPTER 6

SOFTWARE AND HARDWARE DESCRIPTION

6.1 Software Requirements

The system needs some software to obtain sufficient results during the project work. Software description gives the details about the software that is used.

Arduino IDE and Arduino Programming

The Arduino IDE and programming language provide an accessible platform for beginners to get started with microcontroller programming. Its simplicity, extensive library support, and large community make it a popular choice for hobbyists, students, and professionals alike.

6.2 System Requirement

- 1 Processor: Intel core i5 or above.
- 2 64-bit, quad-core, 2.5 GHz minimum per core
- 3 Ram: 4 •GB or more
- 4 Hard disk: 10 GB of available space or more.
- 5 Display: Dual XGA (1024 x 768) or higher resolution monitors
- 6 Operating system: Windows

6.3 Hardware Requirements

Base Platform and Arm Structure:

- Chassis and frame that support all components
- Constructed using 3D-printed PLA parts for the arm's links and a stable mounting board

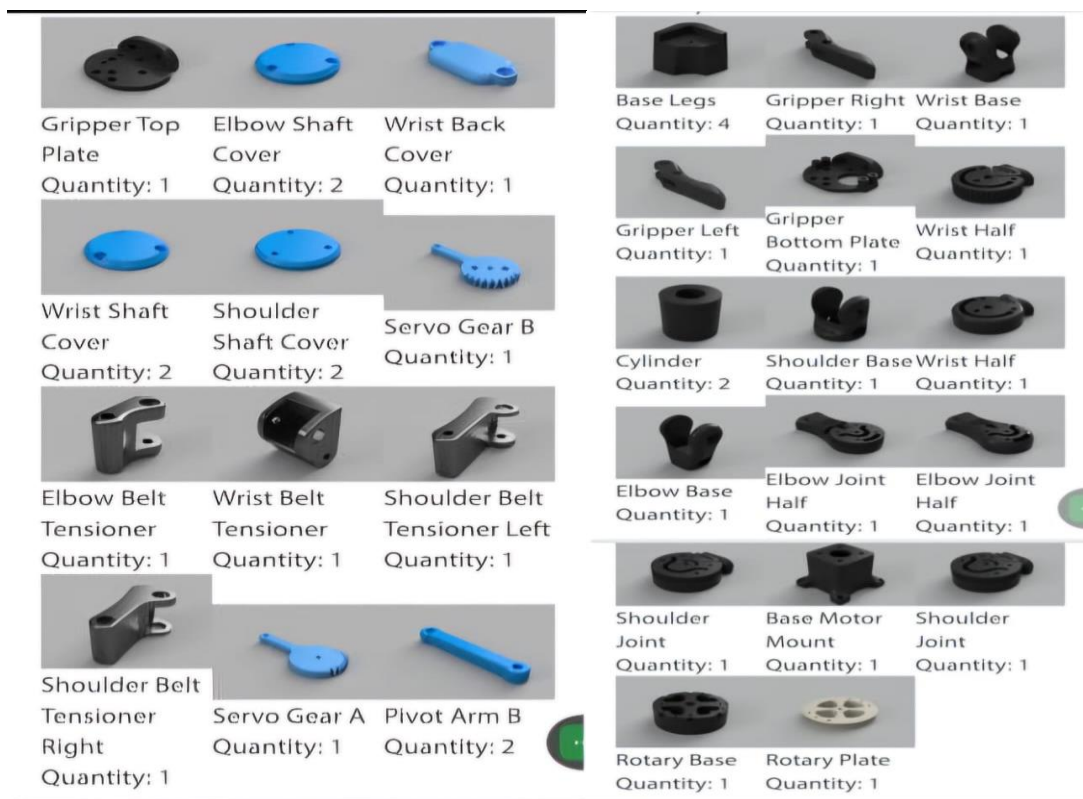


Figure 6.1 PLA+ Parts

Motors:

- Stepper Motors:
 - 2 × NEMA 17 (4.4kgcm)
 - 2 × NEMA 23 (30.6kgcm)
 - 1 × NEMA 14 (2kgcm)
 - 1 × NEMA 17 Planetary Stepper Motor (45kgcm)
- Servo Motor: MG995 for end-effector (gripper) control (12kgcm)

Motor Drivers and Controllers:

- TB6560 stepper motor drivers for interfacing with stepper motors
- RAMPS 1.4 Shield for efficient motor control
- Arduino Mega 2560 as the main controller (powered by the ATmega2560 microcontroller at 16 MHz with ample memory); includes an in-built ESP8266 module for future camera integration and autonomous operation

Power Supply:

- SMPS 24V power supply
- XL4016 DC-DC converter to step down 24V to 12V for motor and board operation

CHAPTER 7

EXPERIMENTAL ANALYSIS AND RESULT

7.1 Result

The 5-DOF pick-and-place robotic arm successfully performed precise movements using stepper motors and a servo motor. By implementing inverse kinematics, the robotic arm accurately positioned its end-effector to pick and place objects at designated locations. The Arduino Mega 2560, in combination with the RAMPS 1.4 shield and TB6560 stepper motor drivers, efficiently controlled the movement of the arm's joints.

7.2 Performance Evaluation

The experimental analysis of the 5-DOF robotic arm using the Arduino Mega 2560, TB6560 stepper drivers, and inverse kinematics algorithms provides valuable insights into its functionality and performance. By integrating Python-based inverse kinematics calculations with Arduino-controlled stepper motor actuation, the robotic arm demonstrates its potential for precise and repeatable motion control. The results of the experiment will contribute to further developments and improvements in the field of robotic manipulators, leading to more efficient and accurate pick-and- place operations in various industrial and research applications.

7.3 Arduino Code

```
#include <AccelStepper.h>
#include <Servo.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define X_STEP_PIN 54
#define X_DIR_PIN 55
#define Y_STEP_PIN 60
#define Y_DIR_PIN 61
#define Z_STEP_PIN 46
#define Z_DIR_PIN 48
#define E0_STEP_PIN 26
#define E0_DIR_PIN 28
#define E1_STEP_PIN 36
#define E1_DIR_PIN 34
AccelStepper stepperX(AccelStepper::DRIVER, X_STEP_PIN, X_DIR_PIN);
AccelStepper stepperY(AccelStepper::DRIVER, Y_STEP_PIN, Y_DIR_PIN);
AccelStepper stepperZ(AccelStepper::DRIVER, Z_STEP_PIN, Z_DIR_PIN);
AccelStepper stepperE0(AccelStepper::DRIVER, E0_STEP_PIN, E0_DIR_PIN);
```

```

AccelStepper stepperE1(AccelStepper::DRIVER, E1_STEP_PIN, E1_DIR_PIN);
AccelStepper* steppers[] = { &stepperX, &stepperY, &stepperZ, &stepperE0, &stepperE1 };
char motorLabels[] = {'x', 'y', 'z', 'p', 'q'};
const float stepsPerDegree = 3200.0 / 360.0;
const float scaleFactorX = 6000.0 / 90.0;
const float scaleFactorY = 450.0 / 90.0;
const float scaleFactorZ = 770.0 / 90.0;
const float scaleFactorQ = 820.0 / 90.0;
const long MAX_STEPS_PER_MOVE = 30000;
const long DEFAULT_X_MAX_SPEED = 1000;
const long DEFAULT_X_ACCEL = 500;
const long DEFAULT_Y_MAX_SPEED = 200;
const long DEFAULT_Y_ACCEL = 100;
const long DEFAULT_Z_MAX_SPEED = 200;
const long DEFAULT_Z_ACCEL = 100;
const long DEFAULT_P_MAX_SPEED = 200;
const long DEFAULT_P_ACCEL = 100;
const long DEFAULT_Q_MAX_SPEED = 200;
const long DEFAULT_Q_ACCEL = 100;
bool emergencyStopActive = false;
char pendingCommand[150];
bool commandPending = false;
struct MotorCommand { AccelStepper* stepper; long steps; };
void emergencyStop() {
    Serial.println("🚨 Emergency Stop Activated!");
    emergencyStopActive = true;
    for (int i = 0; i < 5; i++){
        steppers[i]->stop();
        steppers[i]->setCurrentPosition(0);
        steppers[i]->disableOutputs();
    }
    Serial.println("All motors stopped.");
}
void reinitMotors(){
    Serial.println("Reinitializing motors...");
    for (int i = 0; i < 5; i++){
        steppers[i]->enableOutputs();
        steppers[i]->setCurrentPosition(0);
    }
    emergencyStopActive = false;
    Serial.println("Motors reinitialized. Ready for new commands.");
}
void checkForEmergencyStop() {
    if (Serial.available() > 0) {
        String input = Serial.readStringUntil('\n');
        input.trim();
        if (input.equalsIgnoreCase("s")) { emergencyStop(); }
    }
}
bool parseSubCommandSimultaneous(char* subCmd, MotorCommand &cmd) {
    char* tokens[5];
    int tokenCount = 0;
    char* token = strtok(subCmd, " ");
    while(token != NULL && tokenCount < 5) { tokens[tokenCount++] = token; token = strtok(NULL, " "); }
    if (tokenCount == 0) return false;
    char motor = tokens[0][0];
    AccelStepper* stepper = NULL;
    for (int i = 0; i < 5; i++) { if ((char)tolower(motor) == motorLabels[i]) { stepper = steppers[i]; break; } }
    if (stepper == NULL) { Serial.print("Invalid motor identifier: "); Serial.println(motor); return false; }
    char dir;
    float angle = 0;
    long speed = 0, acceleration = 0;
    float originalAngle = 0;

```

```

if (tokenCount == 2) {
    originalAngle = atof(tokens[1]);
    if (originalAngle < 0) { dir = 'f'; angle = -originalAngle; } else { dir = 'b'; angle = originalAngle; }
    if (tolower(motor) == 'x'){ speed = DEFAULT_X_MAX_SPEED; acceleration = DEFAULT_X_ACCEL; }
    else if (tolower(motor) == 'y'){ speed = DEFAULT_Y_MAX_SPEED; acceleration = DEFAULT_Y_ACCEL; }
    else if (tolower(motor) == 'z'){ speed = DEFAULT_Z_MAX_SPEED; acceleration = DEFAULT_Z_ACCEL; }
    else if (tolower(motor) == 'p'){ speed = DEFAULT_P_MAX_SPEED; acceleration = DEFAULT_P_ACCEL; }
    else if (tolower(motor) == 'q'){ speed = DEFAULT_Q_MAX_SPEED; acceleration = DEFAULT_Q_ACCEL; }
} else if (tokenCount == 5) {
    dir = tokens[1][0];
    originalAngle = atof(tokens[2]);
    angle = (originalAngle < 0) ? -originalAngle : originalAngle;
    speed = atol(tokens[3]);
    acceleration = atol(tokens[4]);
} else { Serial.println("Invalid command format."); return false; }
stepper->setMaxSpeed(speed);
stepper->setAcceleration(acceleration);
long steps = 0;
if (tolower(motor) == 'x') { float effectiveAngle = angle * scaleFactorX; steps = (long)(effectiveAngle * stepsPerDegree); }
else if (tolower(motor) == 'y') { float effectiveAngle = angle * scaleFactorY; steps = (long)(effectiveAngle * stepsPerDegree); }
else if (tolower(motor) == 'z') { float effectiveAngle = angle * scaleFactorZ; steps = (long)(effectiveAngle * stepsPerDegree); }
else if (tolower(motor) == 'q') { float effectiveAngle = angle * scaleFactorQ; steps = (long)(effectiveAngle * stepsPerDegree); }
else { steps = (long)(angle * stepsPerDegree); }
if (dir == 'f') {} else if (dir == 'b') { steps = -steps; } else { Serial.println("Invalid direction."); return false; }
Serial.print((char)tolower(motor));
Serial.print(" motor: original angle = ");
Serial.print(originalAngle, 2);
Serial.print(" → steps = ");
Serial.println(steps);
cmd.stepper = stepper; cmd.steps = steps;
return true;
}

void executeSimultaneousCommands() {
    const int maxCommands = 5;
    MotorCommand commands[maxCommands];
    int count = 0;
    char tempBuffer[150];
    strncpy(tempBuffer, pendingCommand, sizeof(tempBuffer)-1);
    tempBuffer[sizeof(tempBuffer)-1] = '\0';
    char* saveptr;
    char* token = strtok_r(tempBuffer, ",", &saveptr);
    while (token != NULL && count < maxCommands) {
        while (*token == ' ') token++;
        char tokenCopy[50];
        strncpy(tokenCopy, token, sizeof(tokenCopy)-1);
        tokenCopy[sizeof(tokenCopy)-1] = '\0';
        Serial.print("Parsing token: "); Serial.println(tokenCopy);
        if (parseSubCommandSimultaneous(tokenCopy, commands[count])) { count++; }
        token = strtok_r(NULL, ",", &saveptr);
    }
    Serial.print("Total commands parsed: "); Serial.println(count);
    if (count == 0) { Serial.println("No valid commands to execute."); return; }
    for (int i = 0; i < count; i++) { commands[i].stepper->move(commands[i].steps); }
    bool anyRunning = true;
    while (anyRunning && !emergencyStopActive) {
        anyRunning = false;
        for (int i = 0; i < count; i++) { if (commands[i].stepper->distanceToGo() != 0) { anyRunning = true; commands[i].stepper->run(); } }
    }
    checkForEmergencyStop();
}
Serial.println("Simultaneous motor movement completed.");
Serial.println("ACK");
}

```

```

Servo gripper;
const int gripperPin = 11;
const int servoCenter = 90;
int currentRelGripper = -90;
int targetRelGripper = -90;
unsigned long gripperLastUpdate = 0;
const unsigned long gripperStepDelay = 50;
bool gripperMovementActive = false;
void updateGripper() {
    if (emergencyStopActive) { gripperMovementActive = false; return; }
    if (gripperMovementActive) {
        unsigned long currentTime = millis();
        if (currentTime - gripperLastUpdate >= gripperStepDelay) {
            gripperLastUpdate = currentTime;
            if (currentRelGripper < targetRelGripper) { currentRelGripper += 5; if (currentRelGripper > targetRelGripper)
currentRelGripper = targetRelGripper; }
            else if (currentRelGripper > targetRelGripper) { currentRelGripper -= 5; if (currentRelGripper < targetRelGripper)
currentRelGripper = targetRelGripper; }
            int absAngle = servoCenter + currentRelGripper;
            gripper.write(absAngle);
            Serial.print("Gripper moving... Relative: "); Serial.print(currentRelGripper);
            Serial.print("°, Absolute: "); Serial.println(absAngle);
            if (currentRelGripper == targetRelGripper) { gripperMovementActive = false; Serial.print("Gripper target reached. Position:
Relative "); Serial.print(currentRelGripper);
                Serial.print("°, Absolute "); Serial.println(servoCenter + currentRelGripper); }
        }
    }
}
void processGripperCommand(String command) {
    command = command.substring(1);
    command.trim();
    if (command.equalsIgnoreCase("h")) { gripperMovementActive = false; Serial.println("Gripper movement halted."); }
    else { int newTarget = command.toInt(); if (newTarget < -90 || newTarget > 90) { Serial.println("Invalid gripper angle! Use a
value between -90 and 90."); }
        else { targetRelGripper = newTarget; gripperMovementActive = true; Serial.print("Gripper new target set: ");
Serial.print(targetRelGripper); Serial.println(" (relative angle)."); } }
}
void processIKInput(char* input) {
    char temp[100];
    strncpy(temp, input, sizeof(temp)-1);
    temp[sizeof(temp)-1] = '\0';
    float angles[4] = {0};
    int count = 0;
    char* token = strtok(temp, ",");
    while (token != NULL && count < 4) { angles[count++] = atof(token); token = strtok(NULL, ","); }
    if (count != 4) { Serial.println("Invalid IK input. Expecting 4 angles."); return; }
    char angleStr[10];
    strcpy(pendingCommand, "");
    dtostrf(angles[0], 4, 2, angleStr);
    strcat(pendingCommand, "z ");
    strcat(pendingCommand, angleStr);
    strcat(pendingCommand, " ");
    dtostrf(angles[1], 4, 2, angleStr);
    strcat(pendingCommand, "y ");
    strcat(pendingCommand, angleStr);
    strcat(pendingCommand, " ");
    dtostrf(angles[2], 4, 2, angleStr);
    strcat(pendingCommand, "x ");
    strcat(pendingCommand, angleStr);
    strcat(pendingCommand, " ");
    dtostrf(angles[3], 4, 2, angleStr);
    strcat(pendingCommand, "q ");
    strcat(pendingCommand, angleStr);
}

```

```

commandPending = true;
Serial.print("IK joint angles received and buffered: ");
Serial.println(pendingCommand);
Serial.println("Type 'e' to execute the buffered command(s) simultaneously.");
Serial.println("ACK");
}

void setup() {
  Serial.begin(9600);
  delay(2000);
  stepperX.setMaxSpeed(DEFAULT_X_MAX_SPEED);
  stepperX.setAcceleration(DEFAULT_X_ACCEL);
  stepperY.setMaxSpeed(DEFAULT_Y_MAX_SPEED);
  stepperY.setAcceleration(DEFAULT_Y_ACCEL);
  stepperZ.setMaxSpeed(DEFAULT_Z_MAX_SPEED);
  stepperZ.setAcceleration(DEFAULT_Z_ACCEL);
  stepperE0.setMaxSpeed(DEFAULT_P_MAX_SPEED);
  stepperE0.setAcceleration(DEFAULT_P_ACCEL);
  stepperE1.setMaxSpeed(DEFAULT_Q_MAX_SPEED);
  stepperE1.setAcceleration(DEFAULT_Q_ACCEL);
  Serial.println("Stepper Control Commands:");
  Serial.println(" - Type 's' at any time for an emergency stop.");
  Serial.println(" - Buffered commands require pressing 'e' to execute simultaneously.");
  Serial.println(" - Commands can be given as:");
  Serial.println("   Full: <motor> <direction> <angle> <speed> <acceleration>");
  Serial.println("   Manual: <motor> <angle> (angle sign: -ve = forward, +ve = backward)");
  Serial.println("   Multiple commands can be separated by commas.");
  Serial.println(" - IK Input: A comma-separated list of 4 angles (order: Z, Y, X, Q).");
  Serial.println("   Negative values indicate forward movement, positive backward.");
  Serial.println(" - Gripper Commands: Format: g <angle> (e.g., g -90 for fully open, g 20 for fully closed)");
  Serial.println(" - Example Manual command: \"x -90, y 45, z -30, q -20\"");
  gripper.attach(gripperPin);
  currentRelGripper = -90;
  targetRelGripper = -90;
  gripper.write(servoCenter + currentRelGripper);
  Serial.print("Gripper initial position: Relative ");
  Serial.print(currentRelGripper);
  Serial.print("°, Absolute ");
  Serial.println(servoCenter + currentRelGripper);
}

void loop() {
  if (Serial.available() > 0) {
    char inputBuffer[150];
    int n = Serial.readBytesUntil('\n', inputBuffer, sizeof(inputBuffer) - 1);
    inputBuffer[n] = '\0';
    for (int i = 0; i < n; i++) { if (inputBuffer[i] == '\r') { inputBuffer[i] = '\0'; break; } }
    if (strcmp(inputBuffer, "s") == 0 || strcmp(inputBuffer, "S") == 0) {
      emergencyStop();
      gripperMovementActive = false;
      commandPending = false;
      pendingCommand[0] = '\0';
      return;
    }
    bool isIKInput = true;
    for (int i = 0; i < strlen(inputBuffer); i++) { char c = inputBuffer[i]; if (!isdigit(c) && c != '.' && c != '-' && c != '+' && c != 'e' && c != 's' && c != 'S') { isIKInput = false; break; } }
    if (isIKInput && strlen(inputBuffer) > 0) { processIKInput(inputBuffer); return; }
    if (strcmp(inputBuffer, "e") == 0 || strcmp(inputBuffer, "E") == 0) {
      if (commandPending) {
        Serial.print("Executing buffered command(s): ");
        Serial.println(pendingCommand);
        if (emergencyStopActive) { reinitMotors(); }
        executeSimultaneousCommands();
        commandPending = false;
      }
    }
  }
}

```

```

        pendingCommand[0] = '\0';
    } else { Serial.println("No buffered command to execute."); }
} else if (inputBuffer[0] == 'g' || inputBuffer[0] == 'G') {
    processGripperCommand(String(inputBuffer));
} else {
    strncpy(pendingCommand, inputBuffer, sizeof(pendingCommand) - 1);
    pendingCommand[sizeof(pendingCommand) - 1] = '\0';
    commandPending = true;
    Serial.print("Command(s) buffered: ");
    Serial.println(pendingCommand);
    Serial.println("Type 'e' to execute the buffered command(s) simultaneously.");
}
}
}
updateGripper();
}

```

7.4 Python Code For Inverse Kinematics

```

import numpy as np
from scipy.optimize import minimize
import serial
import time
import os

# =====
# Constants and Workspace Limits
# =====
MAX_HEIGHT = 79.7      # cm
MAX_HORIZONTAL_REACH = 55    # cm
BAUD_RATE = 9600        # Must match your Arduino code
SERIAL_PORT = "COM3"     # Update as needed
THETAS_FILE = "current_thetas.txt" # For persisting the current configuration

# =====
# Kinematics Helper Functions
# =====
def deg_to_rad(deg):
    return np.radians(deg)

def dh_transform(theta, a, alpha, d):
    theta_rad = deg_to_rad(theta)
    alpha_rad = deg_to_rad(alpha)
    return np.array([
        [np.cos(theta_rad), -np.sin(theta_rad)*np.cos(alpha_rad),
         np.sin(theta_rad)*np.sin(alpha_rad), a*np.cos(theta_rad)],
        [np.sin(theta_rad), np.cos(theta_rad)*np.cos(alpha_rad), -
         np.cos(theta_rad)*np.sin(alpha_rad), a*np.sin(theta_rad)],
        [0, np.sin(alpha_rad), np.cos(alpha_rad), d],
        [0, 0, 0, 1]
    ])

def forward_kinematics(thetas):
    theta1, theta2, theta3, theta4 = thetas
    theta2_mod = theta2 + 90
    T01 = dh_transform(theta1, 0, 90, 23.5)
    T12 = dh_transform(theta2_mod, 23, 0, 0)
    T23 = dh_transform(theta3, 20.2, 0, 0)

```

```

T34 = dh_transform(theta4, 13, 0, 0)
T04 = T01 @ T12 @ T23 @ T34
return T04[:3, 3]

def is_within_workspace(target):
    x, y, z = target
    radial_distance = np.sqrt(x*2 + y*2)
    if radial_distance > MAX_HORIZONTAL_REACH:
        return False, f"Target exceeds horizontal reach limit of {MAX_HORIZONTAL_REACH} cm."
    if z > MAX_HEIGHT or z < 0:
        return False, f"Target exceeds vertical limits (0 cm <= z <= {MAX_HEIGHT} cm)."
    return True, "Target is within the workspace."

def error_function_continuity(thetas, target, prev_thetas, effective_continuity):
    pos = forward_kinematics(thetas)
    position_error = np.linalg.norm(pos - target)
    continuity_penalty = np.linalg.norm(thetas - prev_thetas)
    return position_error + effective_continuity * continuity_penalty

def inverse_kinematics(target, initial_guess, base_continuity_weight=1.0):
    valid, message = is_within_workspace(target)
    if not valid:
        print(message)
        return None
    current_pos = forward_kinematics(initial_guess)
    pos_error_initial = np.linalg.norm(current_pos - target)
    effective_continuity = (base_continuity_weight / (1 + pos_error_initial)
                           if not np.allclose(initial_guess, [0, 0, 0, 0])
                           else 0.0)
    bounds = [(-180, 180), (-100, 100), (-100, 100), (-100, 100)]
    result = minimize(
        lambda th: error_function_continuity(th, target, np.array(initial_guess), effective_continuity),
        initial_guess,
        method='SLSQP',
        bounds=bounds,
        options={'maxiter': 10000, 'ftol': 1e-6, 'disp': False}
    )
    return result

# =====
# Serial Communication Functions
# =====
# Open a persistent serial connection.
def init_serial():
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=2)
        time.sleep(2) # Allow Arduino to reset
        start_time = time.time()
        while time.time() - start_time < 5:
            if ser.in_waiting > 0:
                line = ser.readline().decode().strip()
                print(f"Discarding: {line}")
                if line == "READY":
                    break
        return ser
    except serial.SerialException as e:
        print(f"Error opening serial port: {e}")
        return None

def send_angles_to_arduino(ser, delta_angles_str):
    message = delta_angles_str + "\n"
    ser.write(message.encode())

```

```

    print(f"Sent delta angles to Arduino: {delta_angles_str}")

# Wait for ACK
ack_received = False
start_time = time.time()
while time.time() - start_time < 5:
    if ser.in_waiting > 0:
        response = ser.readline().decode().strip()
        print(f"Arduino response: {response}")
        if response == "ACK":
            ack_received = True
            break
    if not ack_received:
        print("ACK not received within timeout.")
        return False

    print("Delta angles received by Arduino.")
    # Send execution command for joint/IK movements
    ser.write(b"e\n")
    print("Execution command sent to Arduino.")
    return True

def send_custom_command_to_arduino(ser, cmd):
    message = cmd + "\n"
    ser.write(message.encode())
    print(f"Sent command to Arduino: {cmd}")

    ack_received = False
    start_time = time.time()
    while time.time() - start_time < 5:
        if ser.in_waiting > 0:
            response = ser.readline().decode().strip()
            print(f"Arduino response: {response}")
            if response == "ACK":
                ack_received = True
                break
    if not ack_received:
        print("ACK not received within timeout.")
        return False

    print("Command received by Arduino.")
    # For gripper commands, we do not send an extra execution command.
    return True

# =====
# Persistence for Current Configuration
# =====
def load_current_thetas():
    if os.path.exists(THETAS_FILE):
        try:
            with open(THETAS_FILE, "r") as f:
                line = f.readline().strip()
                thetas = list(map(float, line.split(',')))
                if len(thetas) == 4:
                    return thetas
        except Exception as e:
            print(f"Error reading {THETAS_FILE}: {e}")
    return [0, 0, 0, 0]

def save_current_thetas(thetas):
    try:
        with open(THETAS_FILE, "w") as f:

```

```

        f.write(",".join(map(lambda x: f"{x:.2f}", thetas)) + "\n")
    except Exception as e:
        print(f"Error writing to {THETAS_FILE}: {e}")

# =====
# Global variable for current joint angles
# =====
current_angles = load_current_thetas()

# =====
# Main Loop: IK, Delta, and Commands
# =====
def main():
    global current_angles
    ser = init_serial()
    if ser is None:
        return

    while True:
        print("\nEnter the target end-effector position (x, y, z) in cm, or type:")
        print(" 'r' to refresh (reset previous positions to home),")
        print(" 'h' to move the arm to home position,")
        print(" 'g <angle>' to control the gripper,")
        print(" 'p <angle>' to control the p motor, or")
        print(" 'q' to quit.")
        user_input = input("Your input: ").strip()
        if user_input.lower() == 'q':
            break
        elif user_input.lower() == 'r':
            current_angles = [0, 0, 0, 0]
            save_current_thetas(current_angles)
            print("Current joint configuration reset to home ([0, 0, 0, 0]).")
            continue
        elif user_input.lower() == 'h':
            # Compute delta to home
            delta = np.array([0, 0, 0, 0]) - np.array(current_angles)
            delta_str = f"{delta[0]:.2f},{delta[1]:.2f},{delta[2]:.2f},{delta[3]:.2f}"
            print(f"Commanding move to home position. Delta: {delta_str}")
            if send_angles_to_arduino(ser, delta_str):
                current_angles = [0, 0, 0, 0]
                save_current_thetas(current_angles)
            continue
        elif user_input.lower().startswith("g "):
            # Gripper command
            print("Gripper command received.")
            send_custom_command_to_arduino(ser, user_input)
            continue
        elif user_input.lower().startswith("p "):
            # p motor command
            print("p motor command received.")
            send_custom_command_to_arduino(ser, user_input)
            continue

        # Otherwise, assume a target position is provided:
        try:
            x, y, z = map(float, user_input.split(','))
        except ValueError:
            print("Invalid input format. Please enter three numerical values separated by commas.")
            continue

    target = np.array([x, y, z])

```

```

result = inverse_kinematics(target, initial_guess=current_angles, base_continuity_weight=1.0)
if result is None:
    continue
optimized_thetas = result.x

prev_angles = np.array(current_angles)
new_angles = np.array(optimized_thetas)
delta_angles = new_angles - prev_angles
delta_str =
    f"{delta_angles[0]:.2f},{delta_angles[1]:.2f},{delta_angles[2]:.2f},{delta_angles[3]:.2f}"

expected_base_angle = np.degrees(np.arctan2(y, x))
final_position = forward_kinematics(optimized_thetas)
final_error = np.linalg.norm(final_position - target)

print("\nOptimization terminated successfully:")
print(f"Expected Base Angle (from atan2): {expected_base_angle:.2f}°")
print("Optimized Joint Angles (absolute):")
abs_str =
    f"{optimized_thetas[0]:.2f},{optimized_thetas[1]:.2f},{optimized_thetas[2]:.2f},{optimized_thetas[3]:.2f}"
print(abs_str)
print("Relative Joint Angle Changes (delta from previous):")
print(delta_str)
print(f"Computed End-Effector Position: {final_position}")
print(f"Final Position Error: {final_error:.10f}")

confirm = input("Type 'e' to execute this movement, 's' to stop (skip execution), or any other
    key to cancel and enter a new target: ").strip()
if confirm.lower() == 'e':
    if send_angles_to_arduino(ser, delta_str):
        current_angles = optimized_thetas.tolist()
        save_current_thetas(current_angles)
elif confirm.lower() == 's':
    print("Movement stopped. Returning to target prompt.")
    continue
else:
    print("Execution canceled. Returning to target prompt.")

if __name__ == "__main__":
    main()

```

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

Based on the findings, it is evident that the 5-DOF pick-and-place robotic arm holds great potential for a wide range of industrial and commercial applications by executing precise and reliable operations through advanced motor control and inverse kinematics. Future improvements include making the pick-and-place process fully autonomous by integrating a camera module for real-time object recognition and feedback, which will be controlled through the same Arduino Mega 2560—leveraging its in-built ESP8266 for wireless connectivity. In addition, the system's flexibility will be enhanced by adapting the manipulator for other tasks such as 3D printing, spray painting, sorting, packing, and industrial manufacturing processes like welding and soldering, thereby transforming it into a multi-functional platform. These enhancements will not only improve operational efficiency but also pave the way for further innovations in automation and smart manufacturing.

REFERENCES

1. Adnan Rafi Al Tahtawi, Small-scale Robot Arm Design with Pick and Place Mission Based on Inverse Kinematics published by Journal of Robotics and Control (JRC), Volume 2, Issue 6, in 2021.
2. Noor Saeed , "Forward and Inverse Kinematic Analysis of Robots" published by LAP LAMBERT Academic Publishing in 2020
3. Dilip Kumar Pratihari, "Manipulators, Wheeled and Legged Robots" published by Narosa Publishing House in 2024
4. Jiawei Zhang, "Robot Kinematics: Motion, Kinematics and Dynamics" (2022)
5. Sitangshu Sekhar Biswas, "Hand Book of Bi-Polar Stepper Motor Control" (2021)
6. <https://github.com/MohamedRaslan/UR10PickAndPlace> Simple Pick And Place Matlab Script using UR10 robotic arm in Vrep
7. <https://github.com/PatrickMockridge/Udacity-Kuka-Arm-Pick-And-Place> Programming the rotation and translation matrices for a Kuka KR210 6-DOF arm to perform a basic pick and place operation in the Gazebo robot
8. [https://github.com/Toglefritz/Bertram Robot Arm](https://github.com/Toglefritz/Bertram_Robot_Arm)
9. <https://github.com/BCN3D/BCN3D-Moveo>